



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

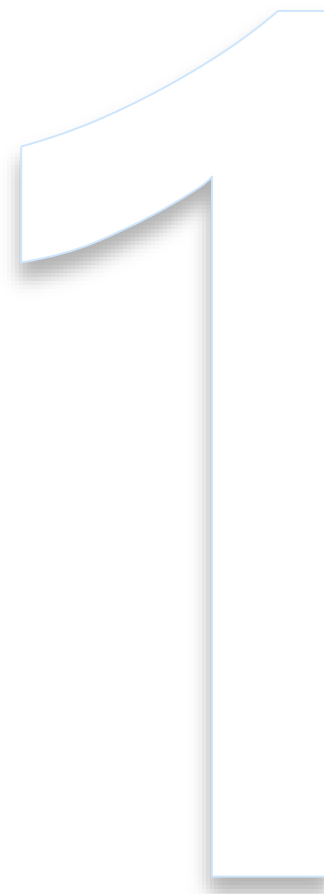
Зэрэгцээ программчлал

Лекц 12

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Түүхэн товчоо
- Зэрэгцээ программчлалын хэлбэрүүд
- Харилцах механизмууд
- Синхрончлолын механизмууд
- Дундын санах ой
- Мессеж солилцоо



Программыг биелэлтийн хувьд,

Хяналтын дарааллыг бүх командыг биелэх эрэмбийн дагуу холбон *урсгал (thread)* гэж төсөөлж болно.

- Дараалсан программ: Зөвхөн нэг урсгал
- Зэрэгцээ программ: *хэд хэдэн идэвхтэй урсгал*

Урсгал гэдгээр ихэнхдээ *процесс, даалгавар* эсвэл *зэрэгцээ программын* тухай ярьдаг ч нэгэн зэрэг идэвхтэй байх тусдаа ажилладаг хэд хэдэн контексттэй тохиолдлыг авч үздэг.

Хэлний хувьд: жнь Ада урсгалын даалгавар (task) гэдэг.

- Ерөнийдөө өөрийн гэсэн хаягийн орон зай бүхий зааврын нийтлэг багцыг **процесс** гэнэ.
 - Үйлдлийн систем шиг
 - Процессыг нөөц (жнь, санах ой) дундаа ашиглаж буй хэд хэдэн урсгалаас бүрдсэн гэж үзнэ.
- Үйлдлийн систем нь
 - хувийн хаягийн орон зай бүхий **хүнд жинтэй (heavyweight)** процесс
 - дундын орон зайг хуваалцах **хөнгөн (lightweight)** процес (урсгал).

Түүхэн товчоо

- Компьютерын ШУ-ы бусад олон чухал чиглэлүүдийн нэгэн адил зэрэгцээ байдал нь техник хангамжийн архитектурын шинэчлэлээс үүдэлтэй
- 1960s device controller: CPU-ийн шууд оролцоогүйгээр data I/O удирдах.
 - CPU нь I/O илгээгээд дуустал хүлээдэг байсан.
 - Төхөөрөмж нь харьцангуй (жнь C/O vs регистр) удаан
 - CPU алдагдал: Карт уншигч 4 card/s, 1960s компьютер 40к/с заавар.
- Тасалдал дээр суурилсан төхөөрөмжийн контроллер ба тэдгээрийн харилцааны механизм нь энэхүү үр ашиггүй байдлыг шийдсэн:
 - CPU нь device controller-т IO команд илгээгээд, бусад заавраа үргэлжлүүлнэ
 - Төхөөрөмж ажиллаж дуусахад контроллер CPU-г тасалдуулах дохио өгдөг.

- Зэрэгцсэн байдал нь програмчлалд шинэ асуудал оруулж, зэрэгцээ байдлыг судалгаа эхэлсэн
 - Жнь **Уралдааны нөхцөл**: Тооцооллын үр дүн нь үйлдлийн эрэмбээс хамаарч дундын өгөгдөлд хандах үед тохиолддог нөхцөл.
- Жнь, дискнээс өгөгдөл уншин тооцоолол хийдэг программ.
 - Программ дагалдах төхөөрөмжөөс өгөгдлийг унших явцад бусад утга дээр тооцооллоо үргэлжүүлдэг байхыг хүснэ.
 - Гэсэн ч дискний дохио хэзээ тооцооллыг тасалдуулахыг таамаглаж чадахгүй:
 - Тасалдал нь санах ойд зарим утгыг ачаалсны дараа тооцооллыг зогсоож, улмаар тасалдал баригч эдгээр утгыг даран бичиж, тооцоолол үргэлжлэхдээ буруу data дээр ажиллаж болно.

ТҮҮХЭН ТОВЧОО» Синхрончлолын хэрэглээ

<i>T1</i>		read x in reg1
<i>T2</i>		read x in reg2
<i>T2</i>		increment reg2
<i>T2</i>		write reg2 in x
<i>T1</i>		increment reg1
<i>T1</i>		write reg1 in x

- T1 ба T2-ын зааврын дараалал: 20 гаруй байхаас 2 нь л зөв бодно.
 - x утгыг 2 болгох: T1/T2-ийг бүтэн, дараа нь нөгөө урсгалыг бүтэн биелүүлэх.

- *Синхрончлол, зэрэгцээ байдлын анхны хэрэгслүүд*
 - mutual exclusion механизм, эгзэгтэй муж (critical section)
 - Зэрэгцээ байдал нь зөвхөн ҮС-ийн түвшинд, хязгаарлагдмал байсан
 - Time-sharing систем: Дээд түвшний хэл дээр зэрэгцээ байдлыг хэрэглэх
- 1960 сүүл: *Зэрэгцээ ажиллах техник, модель.*
 - мультипроцессорын анхны архитектурууд
 - Олон процессор нь өөр өөр програм эсвэл нэг програмын хэсгүүдийг хамтран (эсвэл параллел) гүйцэтгэхэд тусалдаг.
 - Алгоритм, арга зүйн шинэ асуудлууд бий болсон: **параллел программчлал.**
- 1960s: *Компьютерын анхны сүлжээ.*
 - Машиныуд хол зайтай ч нэг нь нөгөөдөө нээсэн замаар “ярилцдаг” зэрэгцээ байдлын өөр нэг модел: *хуваарилагдсан тооцоолол*, дэд бүтцээр дамжих мессеж солилцоонд суурилсан механизмыг ашигладаг.

Зэрэгцээ программчлалын төрлүүд (CP types)

- **Физик зэрэгцээ байдал (*physical concurrency*)**
 - Жнь: хэд хэдэн процессортой машины хувьд хоёр урсгалын (процесс, программ) биелэлт хамтарсан эсвэл параллел байна.
 - Зэрэгцээ ажиллагаа нь цуварсан биелэлтэнд задардаг
- **Логик зэрэгцээ байдал (*logical concurrency*)**
 - Зөвхөн логик түвшний зэрэгцээ байдал
 - Программ нь биелэлтээрээ зэрэгцээ гэж хардаг
- Програмчлалын хэл нь зарчмын хувьд системийн гүйцэтгэлийн үзүүлэлт, биелэх хугацааг тооцохоос бусдаар энэ ялгаа ч ач холбогдолтой биш
 - хэрэв хугацааны T_s нэгжид ажилладаг тухайн асуудлын дараалсан шийдэлтэй, мөн ижил бодлогыг T_p -д ажилладаг зэрэгцээ шийдэлтэй бол физик зэрэгцүүлэлтийн хувьд $T_p < T_s$ байх болно.

- Физик зэрэгцээ програмчлалтай бро ерөнхийдөө *парлел програмчлал*
 - **Өгөгдлийн параллелизм:** нэг үйлдлийг өөр өөр өгөгдөлд хэрэглэдэг
 - **Үйлдлийн параллелизм:** Ижил өгөгдлийн багцад өөр өөр үйлдлүүдийг параллел ашигладаг
- Өөр өөр процессор дээрх ажлын хуваалт
 - Алгоритм, аргууд нь load-balance дэмждэг бол гүйцэтгэлийг үр дүнтэй
 - Тусгай мульти-процессор (вектор процессор г.м) архитектур ашигладаг
 - ШУ-ы тооцооллын салбарт чухал практик хэрэглээтэй: Weather model
- Тусгай алгоритм, арга зүйтэй ч хэл шинжлэл талаасаа = логик зэрэгцээ
 - Дундын СО-той мульти-процессор машинаас - компьютер сүлжээ хүртэлх
 - Синхрончлол: Логик зэрэгцээтэй адил (дундын СО, мессеж солилцоо)

- Нэгээс олон урсгал/процесс байх.
 - 1 машин (1 процессор, дундын санах ойн мульти-процессор) дээр ажиллаж буй нэг программын ижил хугацаанд орших олон идэвхтэй биелэлт.
- Зэрэгцсэн урсгал ба процессуудыг ашиглах хэрэгцээ, шаардлага нь тухайн програмын шийдвэрлэх ёстой асуудлын мөн чанараас хамаарна.
 - Жнь, орчин үеийн веб хөтөч: шинэ хуудас нээх линк орвол шинэ хуудастай холбоотой пакетуудын татах, удирдах, хэлбэржүүлэх (formatting) болон үзүүлэх (магадгүй шинэ урсгалуудыг нээх) гэх мэт шинэ урсгалыг нээдэг.
 - Хэрэглэгч удаан үйлдлийн (зураг харуулах) биелэлтээр илүү хурдан үйлдлийг шийтгэхгүйгээр хөтчийн бусад функцуудтай харилцах боломжтой.
 - Нэг процессортой машинд CPU-г шууд л эзэлдэг (жнь, удаан ажиллагаа дуусахыг хүлээх) эсвэл хэт удаан (бусад тооцоолол үргэжлэхийг хориглоно) урсгалаас зайлсхийхийн тулд хэрэгжүүлэх зарим арга техник хэрэгтэй.

CP TYPES >> Хуваарилагдсан (Distributed) программчлал

- Тархсан физик бүтцээс бүрдсэн машинууд дээр ажиллах зориулалттай.
 - тархсан СО-той мульти-компьютер, янз бүрийн архитектур ба тополог бүхий бодит компьютер сүлжээ.
- Хамгийн чухал:
 - тооцооллын системийн зангилаанууд нь физикийн хувьд тархсан, санах ойгоо хуваалцаж чадахгүй
 - мессеж солилцоонд суурилсан зохих механизм, харилцааны командуудыг нэвтрүүлэх ёстой.

- Зэрэгцээ идэвхжлийн байдлын нь хувьд:
 - доод түвшний заавар,
 - дээд түвшний хэлний командууд,
 - програмын хэсгүүд
 - бүхэлдээ тусдаа програмууд.
- Эхний/сүүлийнх нь зэрэгцээ компиляц, архитектур, харилцах протоколтой илүү холбоотой байна.
 - Нэгж-команд: СО-д дата-г яаж хуваарилах, хэдэн процессор ашиглахыг тусгай тэмдэглэгээ хэрэглэн компиляторт өгдөг уламжлалт дараалсан хэл.
 - Энэ мэдээллийг ашиглан компилятор нь физик машины онцлогуудыг (ихэвчлэн хуваалцсан санах ойтой олон процессор) ашиглах боломжтой оновчилсон параллел объект кодыг гаргадаг.
 - 90s, High Performance Fortran (HPF), data параллелизм бүхий программыг бүтээх боломжтой, ялангуяа ШУ-ы хэрэглээнд ашиглагддаг.

- Зэрэгцээ програмчлалын хүрээнд (төрлөөс үл хамааран) өөр өөр модел, хэлний нийтлэг хоёр тал нь **харилцаа, синхрончлолын** механизм юм.
- Дундын санах ой дээрх харилцаа - shared memory
- Мессеж дамжуулах харилцаа – message exchange
 - **Send / Receive** үйлдэл
 - 2 тал тохиромжтой харилцааны бүтэцтэй бол **channel**

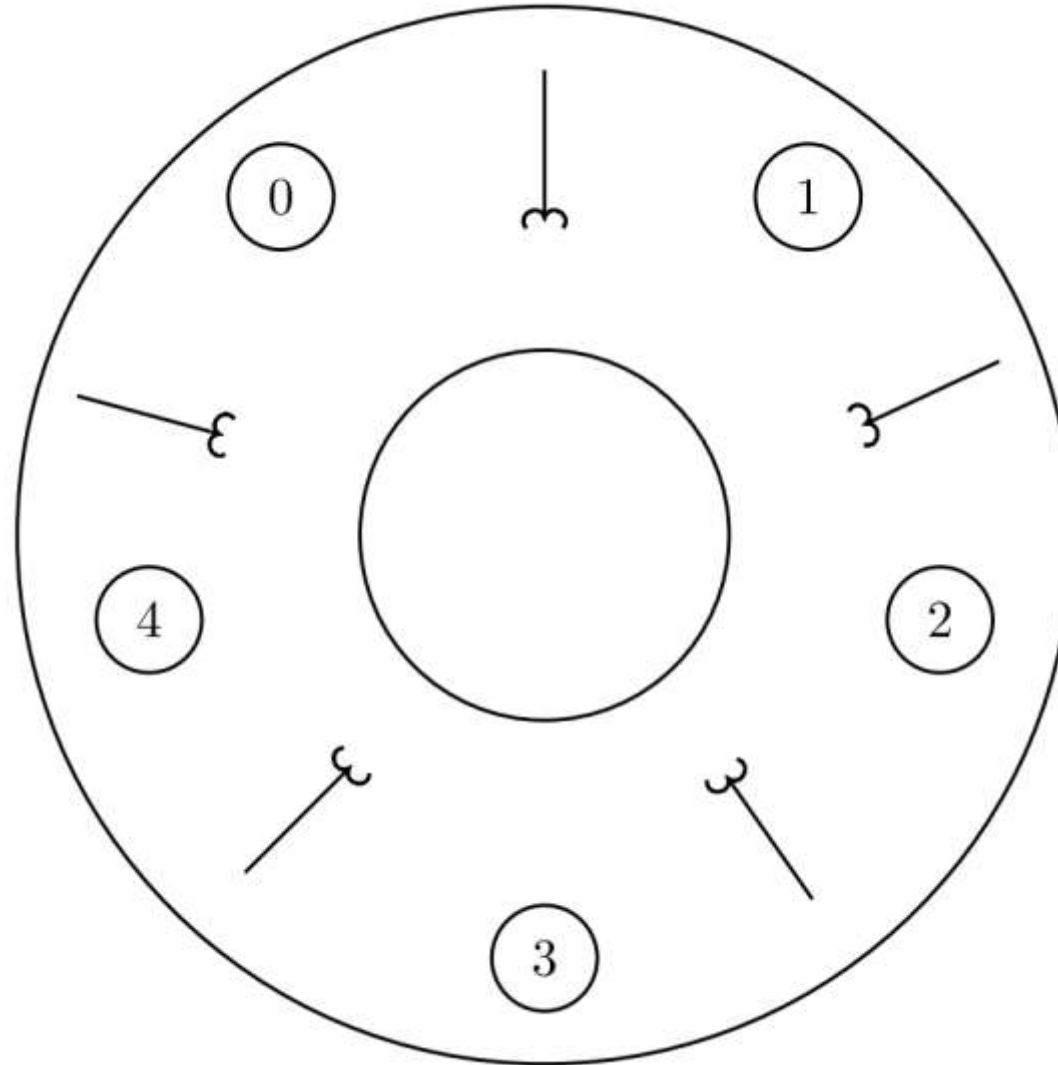
- Дундын санах ойн харилцааны тодорхой бөгөөд синхрончлолын механизмууд тодорхой хэлбэрээр байдаг.
 - mutual exclusion ба condition synchronization
 - **завгүй хүлээх (*busy waiting*)** болон **блок** (мөн хуваарьт-суурилсан синхрончлол гэж нэрлэдэг) ашигладаг.
- Asynchronous Communication
 - Мессеж илгээх, хүлээн авах нь цахим шуудангийн нэгэн адил өөр өөр цаг үед явагддаг: Send үйлдэл Non-Blocking, Receive нь Blocking,
 - Channels хэрэгжүүлэхэд хүндрэлтэй (зарчмын хувьд илгээсэн бөгөөд хараахан хүлээн аваагүй дурын тооны мессеж).
- Synchronous Communication

Дундын санах ой

- Олон урсгал дундын хувьсагч (ижил санах ойн хэсэгт) хандах боломжтой.
 - Харилцаа зөв байхын тулд уралдааны нөхцөл болон бусад буруу байдлаас зайлсхийхийн тулд тусгай синхрончлолын техник хэрэгтэй.
- Busy Waiting
 - Locks
 - Barriers
- Scheduler-Based Synchronization
 - Semaphores
 - Monitors

ДУНДЫН СО Semaphores жишээ

First-In-First-Out (FIFO)



The dining philosophers problem

Мессеж солилцоо

- Хоёр харилцагч хоорондоо "ярилцаж"-ын тулд хэнд мессеж илгээх, хэнээс мессеж хүлээн авахаа мэддэг механизмтай байх ёстой.
 - Талууд өөрсдөө тодорхойлох болох нэрлэх механизм хэрэгтэй.
 - *Илгээгч ба хүлээн авагч* нь процессууд гэж үзнэ
- Naming Mechanisms: Name, Port, Channels
- Asynchronous Communication
- Synchronous Communication
- Remote Procedure Call



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА